Labo 4 - Grid voor de pagina, flexbox voor de rij: vijf nabouw-opdrachten

BIJ	hoofdstuk 4
DUUR	2u
AI	zonder AI (oefening 1 tot 5); thuisopdracht met AI of zonder AI
DOOR	Jeroen De Vos, Kristof Michiels, David Verhulst
CONTEXT	vijf nabouw-opdrachten op flexbox en grid, van drie kolommen tot een volledige pagina; HTML vergrendeld, jij schrijft de CSS; startbestanden in labo-04-starter.zip; pushen is inleveren

Vorige week bouwde je vijf screenshots na met het boxmodel; de rij en de twee helften moest je nog met inline-block tegen elkaar plakken. Deze week krijg je de twee layoutgereedschappen van het moderne web: **flexbox** voor de rij, **grid** voor het raster. Weer vijf nabouwen, in oplopende moeilijkheid: drie kolommen, een navigatiebalk, een Mondriaan, een trap en een volledige pagina voor dierenasiel Zonnehoek. Elke keer een screenshot en een HTML-bestand dat je niet mag wijzigen; elke keer schrijf jij de CSS. Wat je nodig hebt, staat in hoofdstuk 4 van de syllabus, en bij elke oefening stel je eerst de vraag van 4.2: verdeel ik langs één as, of span ik een raster over twee? Aan het einde van dit labo staat de map `labo-04` met vijf werkende nabouwen in je repository.

AI in dit labo

Alle vijf de oefeningen zijn **zonder AI**. Layout is het hart van de toets in november, en layout is precies waar gegenereerde code het vaakst gedateerd is (AI-kader hoofdstuk 4): wie vandaag zelf kiest tussen flexbox en grid, herkent straks in één oogopslag waar een model fout koos. Een concept laten uitleggen mag altijd. De thuisopdracht is **met AI** of **zonder AI**, jouw keuze, mét verantwoording. Elke regel die je indient, moet je in principe mondeling kunnen uitleggen.

Voor je begint

Download `labo-04-starter.zip` van Digitap en pak het uit. Je krijgt een map `labo-04` met een `README.md` en vijf mappen: `kolommen`, `navigatie`, `mondriaan`, `trap` en `dierenasiel`. Elke map bevat `index.html`, `screenshot.png` en een map `css` met `reset.css` en een leeg `stijl.css`. Kopieer `labo-04` naar je map `webtech`, naast de vorige labo's en `site`, en doe *Commit & Sync* met "labo 4 gestart".

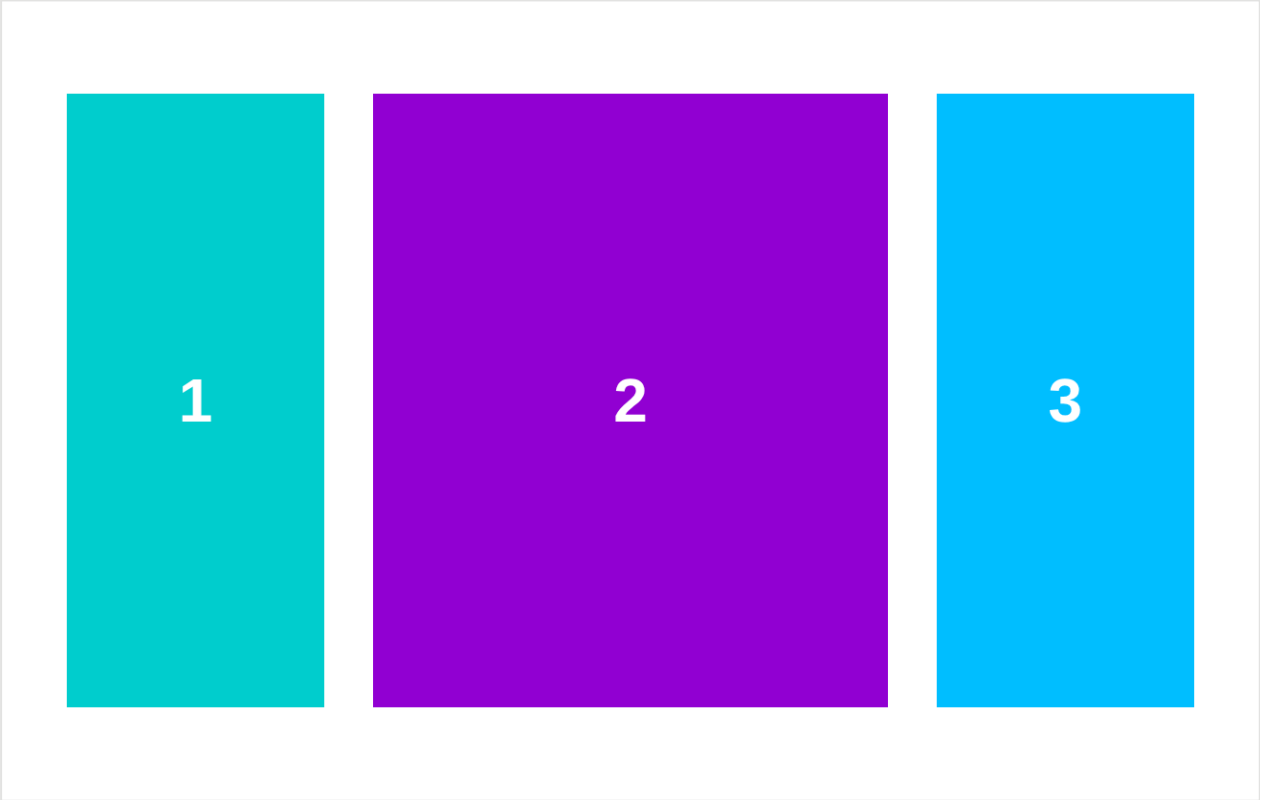
Voor alle vijf de nabouwen gelden dezelfde regels:

- **het HTML-bestand wijzig je niet**: geen class erbij, geen wrapper-div, geen style-attribuut
- `reset.css` staat al gelinkt en wijzig je niet (syllabus 3.3); `stijl.css` begint met een tokenblok op `:root` met de kleuren van de opgave (2.9)
- tussenruimte maak je met `gap`, niet met marges op items (4.3, F4.7); meeteenheden in rem, procent, fr of vh; px alleen voor een rand; geen id-selectors, geen `!important`
- de screenshots zijn genomen in een venster van 1280 bij 800 px; lettertypes verschillen licht per besturingssysteem, kleuren en verhoudingen niet
- lees bij elk screenshot eerst: welke boxen zie ik, wat is de container, en één as of twee? Dan pas typen

Tip: klik in DevTools op de badge *flex* of *grid* naast een element in de Elements-boom (syllabus 4.9). De overlay toont je welke kinderen meedoen en waar de lijnen lopen; het is het snelste antwoord op "waarom staat dit niet waar ik het wil".

1. Drie kolommen

Open `labo-04/kolommen/index.html` met Live Server en schrijf in `css/stijl.css` de CSS die van deze pagina het screenshot maakt. Drie gekleurde kolommen op één rij, de middelste **dubbel zo breed** als de buitenste, met een cijfer in het midden van elke kolom.



Oefening 1: venster van 1280 × 800.

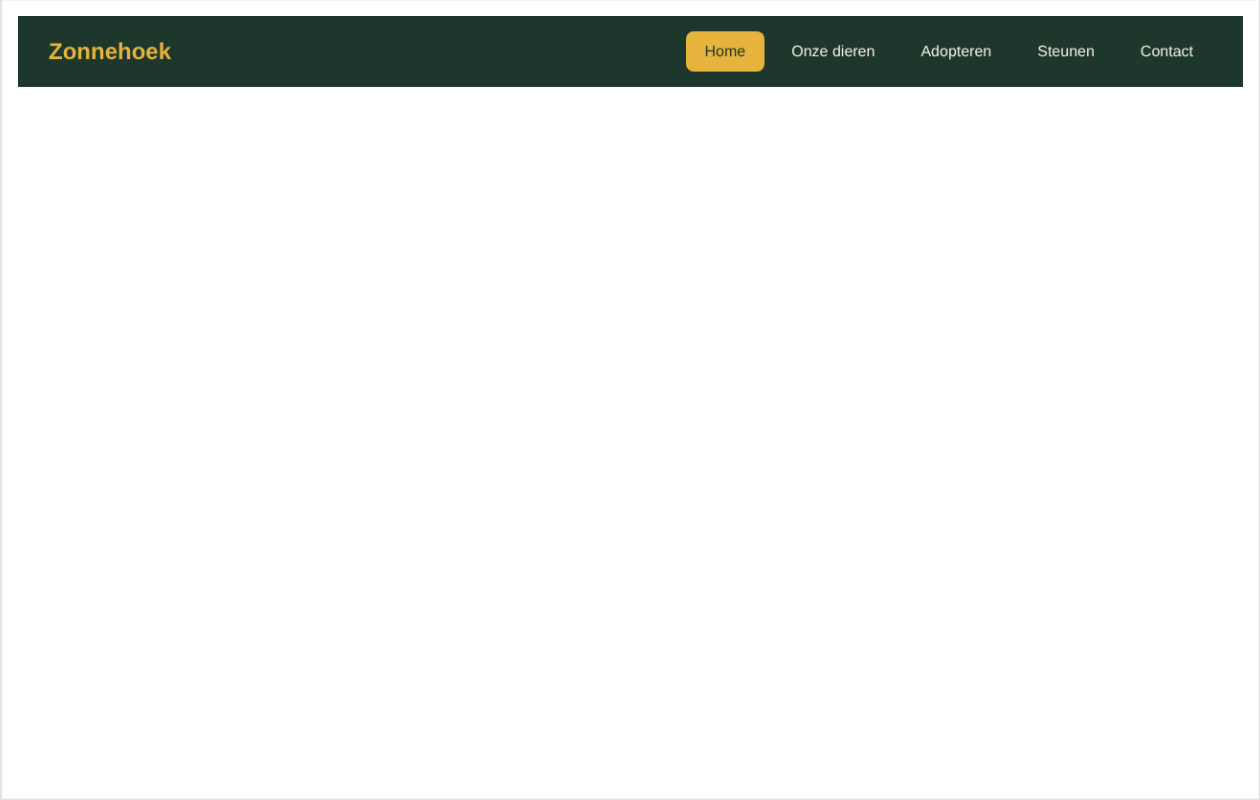
De ontwerper gaf je mee: de kleuren zijn `darkturquoise`, `darkviolet` en `deepskyblue`; de cijfers zijn wit, vet en 4rem groot, in Arial; de rij vult het venster met 10vh ademruimte boven en onder en 4vw links en rechts, en tussen de kolommen zit ook 4vw.

Tip: één as, dus flexbox op `main` (4.3). De verdeling van de breedte regel je op de items met `flex` (4.4): de buitenste krijgen `flex: 1`, de middelste `flex: 2`. Elke kolom is zelf een container om haar cijfer te centreren: het recept van 4.7 werkt ook op een item. De HTML heeft geen classes; selecteer met `:nth-child()` (2.5).

Reflectie (README): op welk element staat `display: flex`, en op welke elementen staat `flex`? Waarom staan die twee niet op hetzelfde element (4.1, F4.2)?

2. Navigatiebalk

Open `labo-04/navigatie/index.html`. De balk van dierenasiel Zonnehoek: de naam links, vijf links rechts op één rij, verticaal gecentreerd, en de actieve link als geel vlak.



Oefening 2: venster van 1280 × 800.

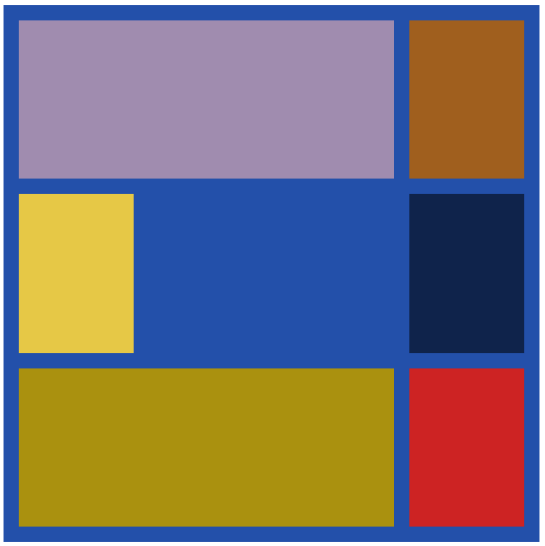
De ontwerper gaf je mee: balk `#1f3a2e`, tekst `#f3efe6`, accent `#e6b53c` (de naam en het vlak achter de actieve link, met de tekst daarop in de balkkleur); Verdana; de balk heeft 1rem boven en onder en 2rem links en rechts; tussen de links zit 0.5rem; elke link is een klikvlak met 0.75rem boven en onder en 1.25rem links en rechts, en ronde hoeken van 0.5rem.

Tip: twee flex-containers (4.3). De `header` zet naam en navigatie uit elkaar met `justify-content: space-between` en lijnt ze verticaal uit met `align-items: center`; de `ul` is een rij met `gap`. De lijst brengt haar user-agent-bagage mee (3.3). De links zijn inline, en een klikvlak vraagt een box (3.7, F3.5). Hover en focus: zie de beschrijving in `index.html`.

Reflectie (README): welke property verdeelt de ruimte langs de hoofdas, en welke lijnt uit op de kruisas (4.3)? Wat gebeurt er met de links als je `gap` vervangt door `margin-right` op elke `li` (F4.7)?

3. Mondriaan

Open `labo-04/mondriaan/index.html`. Een vierkant schilderij van zes vlakken op een blauw raster: vier kolommen, drie rijen, en het blauw is niets anders dan de **gap** en de padding van de container die doorheen de vlakken schijnt. Twee cellen in het midden blijven leeg.



Oefening 3: venster van 1280 × 800.

De ontwerper gaf je mee: blauw `#2554ab`; de vlakken `#a090af` (lila), `#a16122` (bruin), `#e7c948` (geel), `#13234e` (marine), `#ae9310` (olijf) en `#cd2424` (rood), elk met de class van hun kleur; het schilderij is 70vh breed en hoog en staat horizontaal gecentreerd met 15vh boven en onder; de lijnen zijn 2vh dik, ook aan de buitenrand.

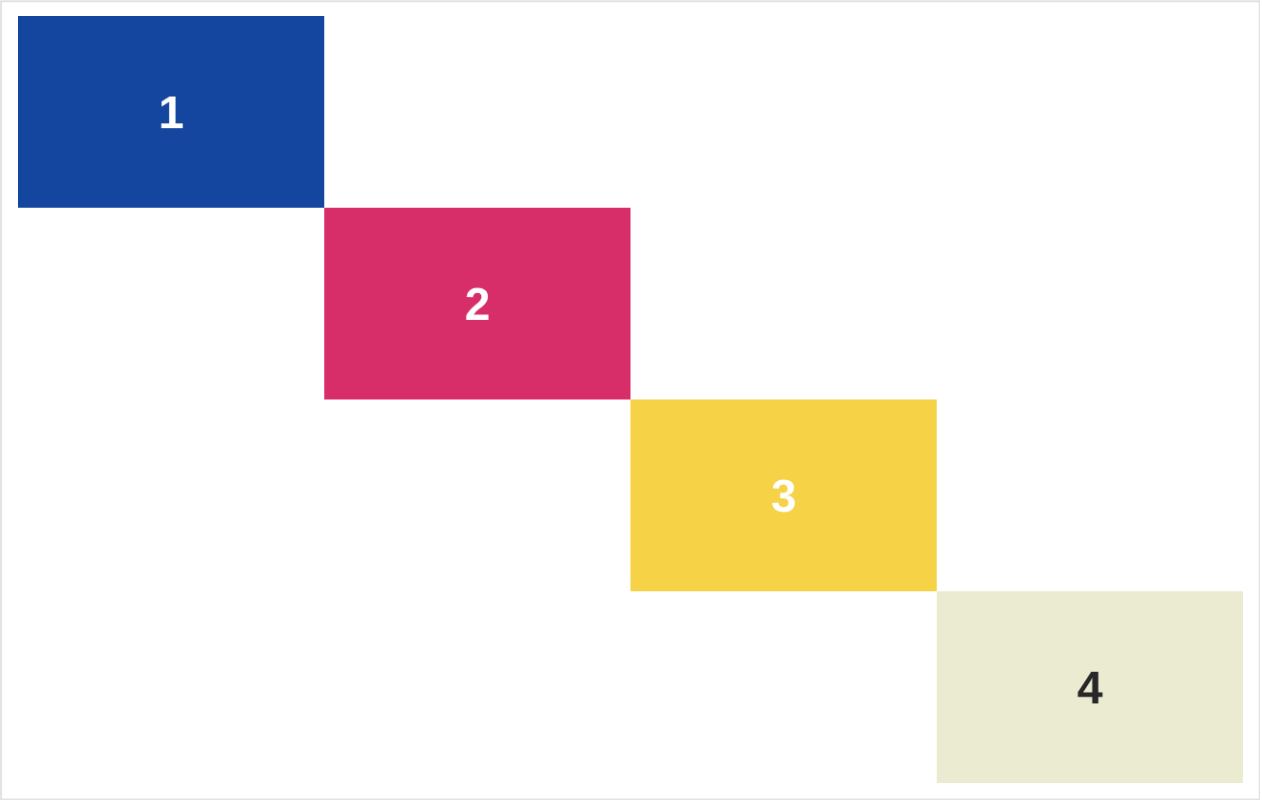
Tip: twee assen, dus grid (4.5): `repeat(4, 1fr)` voor de kolommen en drie rijen die elk een gelijk deel van de hoogte krijgen. Plaats de brede vlakken met `grid-column` van lijn tot lijn (4.6); de rest vult vanzelf. Tel de lijnen op het screenshot vóór je typt: een raster van vier

kolommen heeft vijf kolomlijnen. De lege cellen maak je niet, die ontstaan door wat je wél plaatst.

Reflectie (README): welke twee vlakken moest je expliciet plaatsen, en waarom volstond de impliciete vulling voor de andere vier (4.5, 4.6)? Waarom is `gap` hier beter dan een border op elk vlak?

4. Trap

Open `labo-04/trap/index.html`. Vier treden die het venster diagonaal doorkruisen: een raster van vier bij vier cellen dat het hele venster vult, en elke trede in haar eigen cel op de diagonaal.



Oefening 4: venster van 1280 × 800.

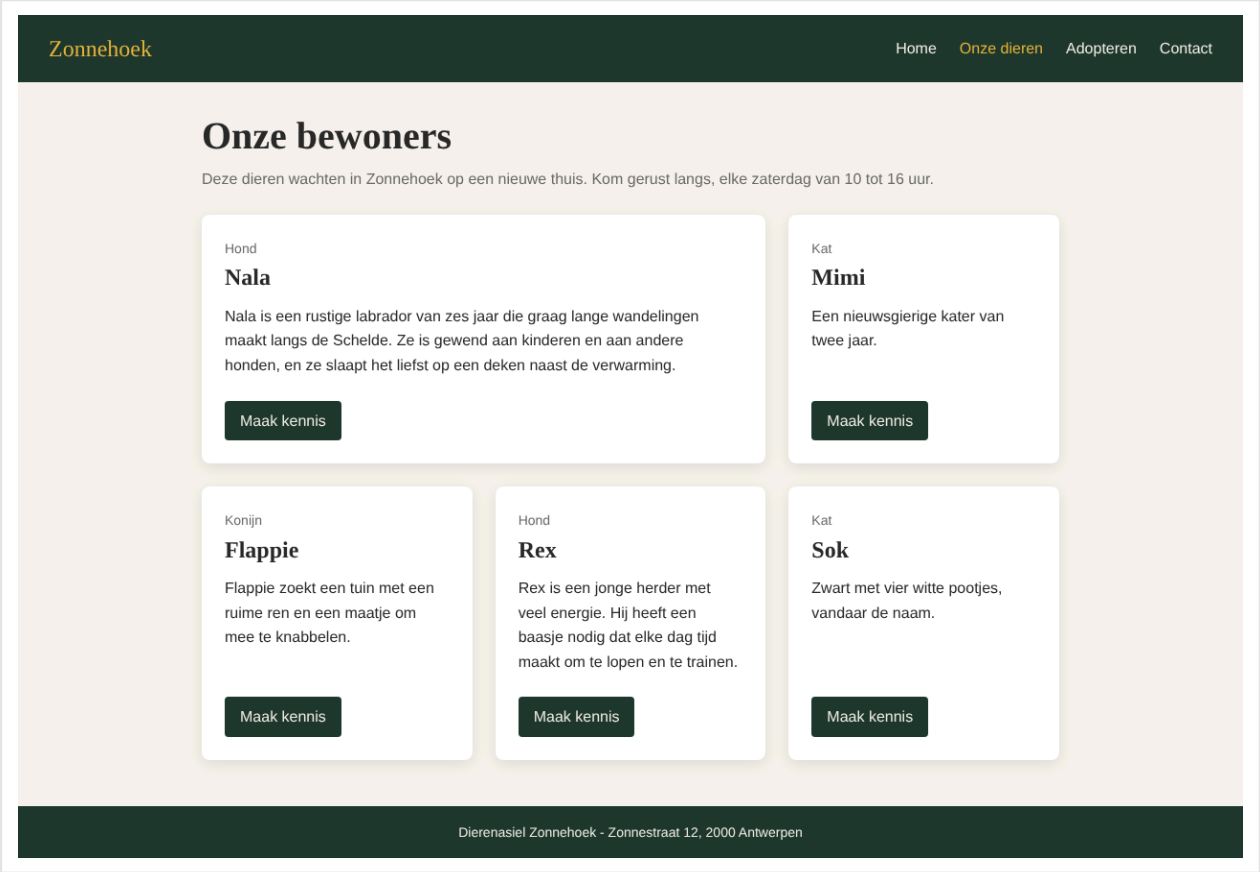
De ontwerper gaf je mee: de treden zijn `#1446a0`, `#db3069`, `#f5d547` en `#ebebd3`, met een wit cijfer van 3rem in het midden (op de laatste, lichte trede is het cijfer `#2b2b2b`); Arial; het raster vult het venster volledig, zonder tussenruimte.

Tip: zonder plaatsing zet de impliciete grid de vier treden naast elkaar op de eerste rij (4.5). Elke trede krijgt dus haar eigen `grid-column` én `grid-row` (4.6), en omdat de HTML vier keer dezelfde class heeft, selecteer je per trede met `:nth-child()`. Het raster zelf hangt aan het venster: `100vh` (3.2).

Reflectie (README): wat zie je als je de `grid-row`-declaraties weghaalt, en waarom (4.5)? Wat betekent `grid-column: 4 / -1` voor de laatste trede, en waarom werkt het hier ook (motorkap hoofdstuk 4)?

5. Dierenasiel Zonnehoek

Open `labo-04/dierenasiel/index.html`. Alles samen: het grondplan van een pagina met de footer onderaan, de navigatiebalk van oefening 2, een leeskolom, en een rooster van vijf kaarten waarvan de eerste twee kolommen breed is. In elke kaart staat de knop onderaan, hoe kort of lang de tekst ook is.



Oefening 5: venster van 1280 × 800, volledige pagina.

De ontwerper gaf je mee: papier `#f7f4ec`, kaart wit, inkt `#2b2b2b`, gedempt `#6b6b6b`, balk en knop `#1f3a2e` met tekst `#f3efe6`, accent `#e6b53c` (de naam en de actieve link); tekst in Verdana met regelhoogte 1.6, koppen in Georgia; de leeskolom is hoogstens 60rem breed met 2rem padding; tussen de kaarten 1.5rem, en elke kaart heeft 1.5rem padding, ronde hoeken van 0.5rem en een zachte schaduw (3.5); de knop heeft 0.5rem boven en onder, 1rem links en rechts en hoeken van 0.25rem.

Tip: het bouwplan van 4.8, van buiten naar binnen. `body` is een grid van drie rijen, `auto 1fr auto`, met `min-height: 100vh` (4.7). De `header` is de flex-balk van oefening 2. `.kaarten` is een grid van drie kolommen met `gap`, en `.uitgelicht` loopt van lijn 1 tot lijn 3 (4.6). Elke

`.kaart` is een flex-container in richting column; geef `.beschrijving` `flex: 1` en de knop staat onderaan (4.4). Zet `align-items: flex-start` op de kaart, anders rekt de knop zich uit tot de volle breedte (F4.5). Hover en focus: zie de beschrijving in `index.html`.

Reflectie (README): hoeveel layoutcontainers telt deze pagina, en welk gereedschap koos je voor elk? Verklein het venster tot de helft: wat breekt er, en in welk hoofdstuk wordt dat opgelost?

Klaar?

De overlay. Klik in elke nabouw de badge van elke container aan (4.9): zijn dit de items die je bedoelde, en lopen de lijnen waar je ze dacht? Geen waarschuwingsteken bij je eigen declaraties (F2.2, F2.3).

De regels. Vijf keer: `reset.css` gelinkt en ongewijzigd, tokenblok op `:root`, `gap` in plaats van marges tussen items, geen px buiten randen, geen id-selector, geen `!important`, en `index.html` ongewijzigd.

README. Alle reflecties staan in `labo-04/README.md`, met je naam bovenaan.

Inleveren. *Commit & Sync* met "labo 4: flexbox en grid". Is alles gepusht, dan is het ingeleverd. De feedback-tooling controleert of de HTML ongewijzigd is, of je stylesheets de regels volgen, en meet per nabouw een aantal boxen in de browser; wat ze vindt, lees je in de loop van volgende week, met verwijzing naar de sectie of het foutnummer.

Thuis

Werk de nabouwen af die in de sessie niet lukten. Doe daarna B4.1 uit de syllabus voor je site, **met AI** of **zonder AI**: het grondplan met sticky footer op `body`, de navigatie als flex-rij, een kaartenrooster voor je projecten of interesses in grid, en elke kaart als flex-column met de knop onderaan. Elke container in je CSS krijgt een commentaarregel met zijn taak. Kies je de AI-route, bewaar dan je prompt en de onbewerkte output in `site/review/` en schrijf in je README wat je corrigeerde: floats of absolute positioning tussen het moderne, flexbox waar een raster hoorde, wrapper-lasagne, pixelrasters (AI-kader hoofdstuk 4). Push met "site: layout". Lees voor de theorieles 5.1 tot en met 5.3 van de syllabus.